

미션 17 보고서 - MNIST ONNX 손글씨 숫자 인식 서비스

팀명/이름: 4팀 김도민

Docker Hub URL: <https://hub.docker.com/r/ming2tofu33/mission17-mnist-streamlit>

Docker image: ming2tofu33/mission17-mnist-streamlit:latest

1. 프로젝트 개요

이번 프로젝트는 사용자가 Streamlit 웹 화면에서 손글씨 숫자를 그리면 MNIST ONNX 모델을 이용해 0부터 9까지의 숫자 중 어떤 숫자인지 예측하는 AI 서비스입니다. 모델을 새로 학습하는 것이 아니라, 준비된 ONNX 모델을 실제 사용자가 입력하고 결과를 확인할 수 있는 웹 서비스로 연결하는 데 목적이 있습니다.

2. 서비스 화면 구성

화면은 입력 캔버스, 전처리 이미지, 모델 추론 결과, 이미지 저장소 영역으로 구성했습니다. 입력 캔버스에서는 사용자가 숫자를 직접 그리고, 전처리 이미지 영역에서는 모델 입력에 맞게 바뀐 28x28 흑백 이미지를 확인합니다. 모델 추론 결과 영역에서는 최종 예측 숫자와 class별 확률 차트를 보여주며, 이미지 저장소에는 추론에 사용된 이미지와 예측 결과가 저장됩니다.

3. 모델과 추론 흐름

MNIST CNN ONNX 모델을 models/mnist_cnn.onnx 경로에 두고 ONNX Runtime으로 로드했습니다. Streamlit 캐싱 기능을 사용해 모델 세션을 한 번 생성한 뒤 재사용하도록 구성했습니다.

사용자가 그린 캔버스 이미지는 RGBA 형태일 수 있으므로 흰 배경에 합성한 뒤 흑백으로 변환합니다. 그 다음 숫자 영역을 중심으로 잘라 28x28 크기로 조정하고, 픽셀 값을 0부터 1 사이의 float32 값으로 정규화합니다. 최종 입력 shape는 모델이 요구하는 (1, 1, 28, 28)입니다.

4. 코드 구조

파일	역할
app.py	Streamlit 화면 구성, 캔버스 입력, 모델 추론, 결과 표시, 이미지 저장소 표시
image_processing.py	빈 캔버스 확인, 이미지 전처리, 전처리 이미지 저장
inference_utils.py	ONNX 세션 생성, logits를 확률로 변환, 최종 예측 label 계산
tests/	전처리와 추론 결과 해석 로직 자동 테스트
Dockerfile	Streamlit 서비스를 Docker 이미지로 실행하기 위한 설정

5. Docker 배포

Dockerfile을 작성해 Streamlit 앱, 전처리 코드, ONNX 모델 파일, 필요한 Python 라이브러리를 하나의 Docker 이미지로 묶었습니다. 빌드한 이미지는 Docker Hub의 ming2tofu33/mission17-mnist-streamlit 저장소에 latest 태그로 업로드했습니다.

로컬 실행 확인 URL: <http://127.0.0.1:8501>

Docker 컨테이너 테스트 포트: <http://127.0.0.1:8502>

6. 디버깅과 해결 과정

항목	내용
----	----

오류 상황	캔버스 이미지를 그대로 모델에 넣으면 입력 shape와 색상 형식이 맞지 않을 수 있음
원인 추정	캔버스 출력은 RGBA 이미지이고, MNIST 모델은 (1, 1, 28, 28) float32 입력을 기대함
해결 방법	흑백 변환, 색상 반전, 중심 crop, 28x28 resize, 정규화, shape 변환을 전처리 함수로 분리함
확인 결과	pytest 자동 테스트 4개 통과, 실제 ONNX 세션 추론 성공, Streamlit과 Docker 실행 HTTP 200 확인

7. 실행 및 검증 결과

자동 테스트: `pytest -q` 실행 결과 4 passed

ONNX 추론 검증: 모델 입력 shape (1, 1, 28, 28), 확률 합계 1.0 확인

Streamlit 로컬 검증: `http://127.0.0.1:8501` HTTP 200 응답 확인

Docker 검증: `docker run` 후 `http://127.0.0.1:8502` HTTP 200 응답 확인

Docker Hub push: latest 태그 업로드 성공